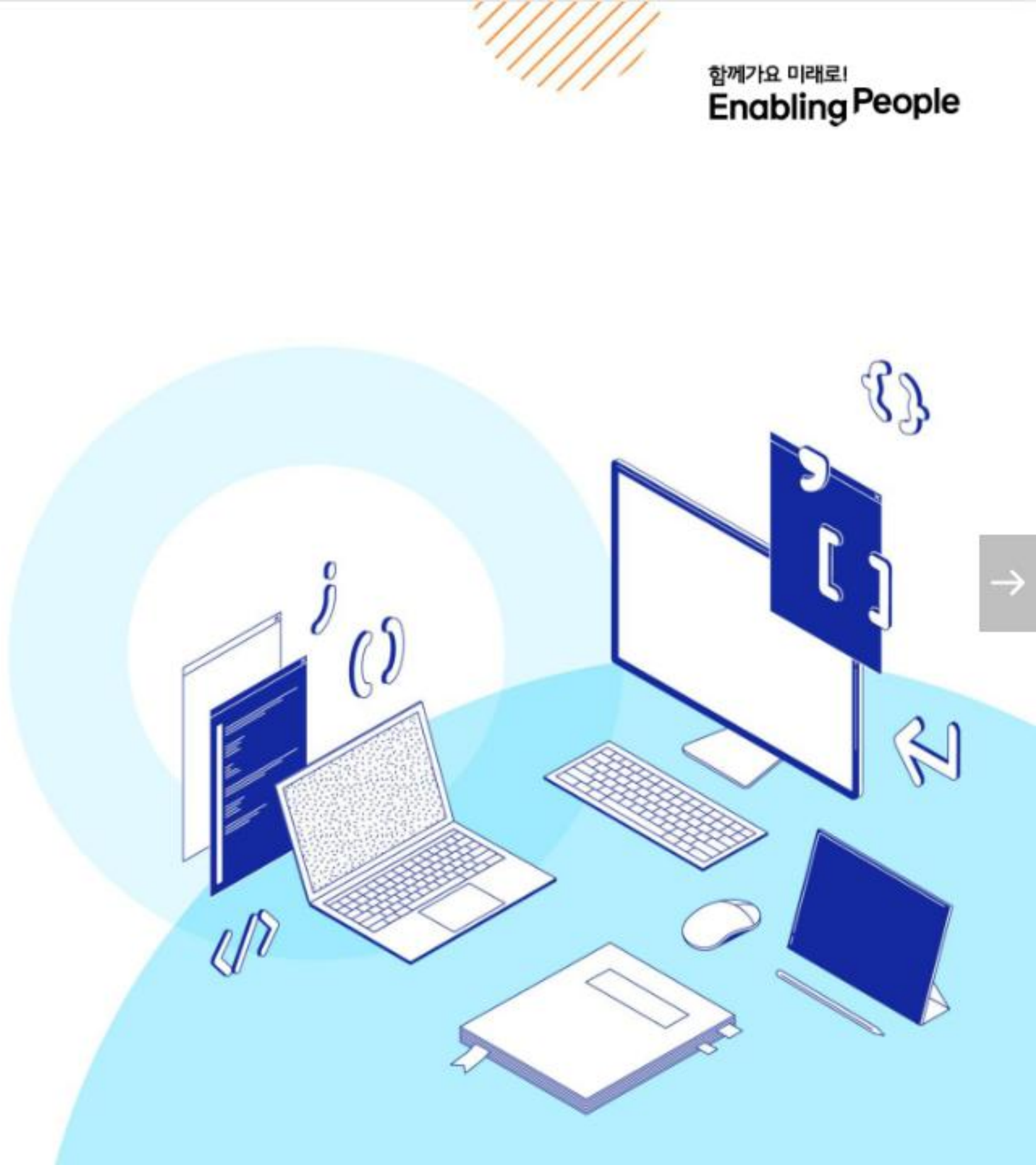


SSAFY

Database

Many To One Relationships 01

Python



목 차

Many to one relationships

- 모델 관계
- 댓글 모델 정의
- 댓글 생성 연습

관계 모델 참조

- 역참조

댓글 구현

- 댓글 CREATE
- 댓글 READ
- 댓글 DELETE

목 차

참고

- 데이터 무결성
- admin site 댓글 등록
- 댓글 추가 구현



학습 시작

사내 게시판에 작성된 게시글에 댓글을 작성할 수 있도록 만들어 봅시다.

- 사용자가 게시글에 댓글을 달 수 있는 기능을 Django로 구현해야 하는 상황입니다.
- 게시글과 댓글을 데이터베이스에 저장을 해야합니다.

1. 게시글과 댓글은  관계이므로, 모델 간 관계 설정이 필요해  를 사용해 댓글 모델을 정의해야 합니다.
2.  개념을 활용하면 게시글에서 관련된 댓글을 쉽게 조회할 수 있습니다.
3.  문법을 사용하면 댓글이 없는 경우에도 자연스러운 UI를 구현할 수 있습니다.

- admin 페이지에 댓글을 등록하고 관리하는 방법도 배웁니다.
- Django에서 모델 관계를 기반으로 기능을 설계하고 확장할 수 있는 역량을 키울 수 있습니다.

학습 목표

- ✓ Django의 Many to One 모델 관계를 이해하고 정의할 수 있다.
- ✓ ForeignKey를 활용한 관계형 모델 구축 방법을 실습할 수 있다.
- ✓ Django shell을 이용해 댓글 데이터를 생성하고 관계를 확인할 수 있다.
- ✓ CREATE/READ/DELETE 기능을 사용해 댓글 기능을 Django 프로젝트에 구현할 수 있다.
- ✓ 역참조를 통해 관계 모델에서 데이터를 효율적으로 조회할 수 있다.
- ✓ admin 페이지에서 댓글 관리를 수행할 수 있다.
- ✓ for-empty 문법을 활용해 사용자 경험을 개선할 수 있다.

Many to one relationships

모델 관계

관계(relationship)의 정의

관계

relationship

데이터베이스 내 여러 테이블 간의
논리적인 연결 관계를 나타냄

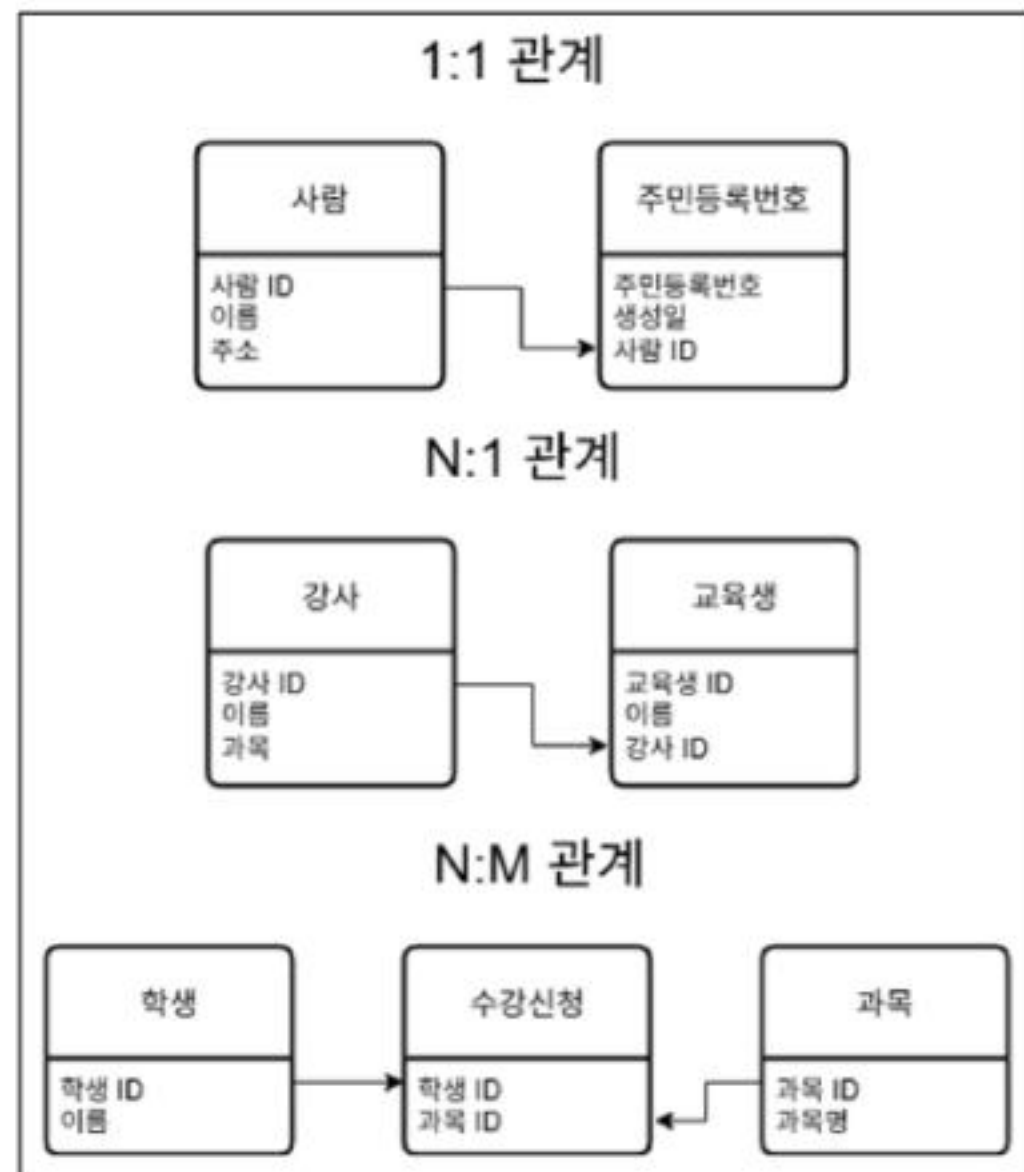
데이터베이스에는 '강사' 테이블과 '학생' 테이블이 있을 때, “이삼성 강사가 김싸피 학생을 가르치고 있다”는 정보를 저장하려면 두 테이블 간의 관계(relationship)를 설정해야 합니다.

이러한 관계를 표현하기 위해, 한 테이블에 다른 테이블의 기본 키(Primary Key)를 저장하며, 이처럼 다른 테이블의 기본 키를 참조하는 속성을 외래 키(Foreign Key)라고 합니다.

외래 키는 두 테이블 간의 논리적인 연결 고리 역할을 하며, 이를 통해 관계형 데이터베이스에서 연관된 데이터를 효율적으로 관리할 수 있습니다.

관계의 종류

- 1:1 (One to One) 관계
 - 한 테이블의 레코드는 다른 테이블의 한 레코드와 연결됨
 - 예시) 한 사람당 하나의 주민등록번호
- N:1 (Many to One) 관계
 - 여러 개의 레코드가 하나의 레코드와 연결됨
 - 예시) 여러 교육생(N)을 한 강사(1)가 가르침
- N:M (Many to Many) 관계
 - 여러 레코드가 여러 레코드와 상호 연결됨
 - 보통 중간 테이블(예: 수강신청) 을 사용해 구현됨
 - 예시) 여러 학생(N)이 여러 과목(M)을 수강함



<그림1_DB_Many To One Relationships 01_관계의 종류>

※ 1:1 관계는 N:1 관계와 사용방법은 유사하며, N:M 관계는 추후 진행될 예정

 Many to one relationships

Many to one relationships

N:1 or 1:N

한 테이블의 0개 이상의 레코드가
다른 테이블의 레코드 한 개와 관련된 관계

Many to one relationships 예시 (1/2)

- SSIFY Track : Student
 - SSIFY Track과 교육생의 관계?
- Account : Bank
 - 계좌와 은행의 관계?
- Baseball Team : Member
 - 야구 팀과 야구 선수의 관계?
- Order : Customer
 - 주문과 고객의 관계?
- Country : City
 - 국가와 도시의 관계?
- Movie : Actor
 - 영화와 배우의 관계?

Many to one relationships 예시 (2/2)

- SSIFY Track (1) : Student (N)
 - 하나의 트랙에는 여러 명의 교육생이 포함됨
 - 한 명의 교육생은 트랙이 중복되어서는 안됨
- Account (N) : Bank (1)
 - 하나의 계좌는 한 은행에만 소속되어 있음
 - 한 은행은 여러 개의 계좌를 가질 수 있음
- Baseball Team (1) : Member (N)
 - 한 팀에 여러 명의 선수가 소속됨
 - 한 선수가 다른 야구 팀에 소속될 수 없음
- Order (N) : Customer (1)
 - 한 명의 고객이 여러 주문 가능
 - 하나의 주문을 여러 명의 고객이 낼 수 없음
- Country (1) : City (N)
 - 하나의 국가는 여러 도시를 가짐
 - 한 도시가 중복된 국가에 포함되지 않음
- Movie (N) : Actor (M) => N:1 관계 아님
 - 영화에 여러 배우들이 출연 가능함
 - 배우도 여러 영화에 출연 가능함

댓글과 게시글의 관계

Comment(N) : Article(1)

하나의 게시글에는 여러 개의 댓글이 달릴
단, 게시글에 댓글이 없는 경우도 존재함
댓글이 소속된 게시글이 없는 경우는 없음

- 즉, 0개 이상의 댓글은 1개의 게시글에 작성될 수 있다.

테이블 관계 설정 (1/2)

- 관계 설정을 위한 Foreign Key(외래 키, FK)를 N:1에서 1을 담당하는 테이블에 위치하면 안됨
 - Article Table에 Foreign Key 컬럼을 위치시키면 중복 데이터로 인해 낭비가 발생함

Comment Table
id
content
created_at
updated_at

<표1_DB_Many To One Relationships 01_1의 테이블에 외래 키 설정 1>

Article Table
id
title
content
created_at
updated_at
Comment에 대한 정보(외래 키)

<표2_DB_Many To One Relationships 01_1의 테이블에 외래 키 설정 2>

※ 댓글 생성마다 Comment의 정보와 함께 Article 정보(id, title, content, ..)가 매 번 중복 저장됨

테이블 관계 설정 (2/2)

- 관계 설정을 위한 Foreign Key(외래 키, FK)의 적절한 위치는 바로 N:1에서 N을 담당하는 테이블에 위치
 - Comment가 생성되면 Article의 정보만 저장하면 됨

Comment Table
id
content
created_at
updated_at
Article에 대한 정보(외래 키)

<표3_DB_Many To One Relationships 01_N의 테이블에 외래키 설정 1>

Article Table
id
title
content
created_at
updated_at

<표4_DB_Many To One Relationships 01_N의 테이블에 외래키 설정 2>

※ 외래 키 컬럼에 저장되는 데이터는 참조하는 데이터를 대표하는 Primary Key(PK) 정보를 저장함

댓글 모델 정의

ForeignKey 필드

ForeignKey(to, on_delete)

한 모델이 다른 모델을 참조하는 관계를 설정하는 필드

- N:1 관계 표현할 때 사용
- 데이터베이스에서 외래 키로 구현됨

ForeignKey 필드

ForeignKey(to, on_delete)



데이터 무결성
데이터가 정확하고 일관되며,
신뢰할 수 있도록 유지되는 상태

<개념1_DB_Many To One Relationships 01_데이터 무결성>

- to 속성

- 참조하는 모델 class 이름 (N:1에서 N이 아닌 1의 class 정보)

- on_delete 속성

- 외래 키가 참조하는 객체(1)가 사라졌을 때, 외래 키를 가진 객체(N)를 어떻게 처리할 지를 정의하는 설정 (데이터 무결성)

※ to와 on_delete 속성은 ForeignKey 설정에 필수 요소

on_delete 속성 종류

- CASCADE

- 참조된 객체(부모 객체)가 삭제될 때 이를 참조하는 모든 객체도 삭제되도록 지정
- 예) 게시글이 삭제되면 해당 게시글의 모든 댓글을 삭제

- PROTECT

- 삭제하려는 부모 객체에 자식 객체가 존재한다면 해당 부모 객체를 삭제하지 못하도록 지정
- 예) 게시글을 삭제할 때 해당 게시글에 댓글이 존재하면 게시글 삭제 불가

- SET_NULL

- 부모 객체가 삭제되면, 해당 필드에 값이 NULL이 저장되도록 지정
- 단, 해당 ForeignKey필드 설정이 `null = True`가 설정 되어야 함

- 기타 on_delete 설정 값 참고 페이지

- <https://docs.djangoproject.com/en/5.2/ref/models/fields/#arguments>

댓글 모델 정의하기

- ForeignKey 클래스의 인스턴스 이름은 참조하는 모델 클래스 이름의 **단수형**으로 작성하는 것을 권장

```
# articles/models.py
class Comment(models.Model):
    article = models.ForeignKey(Article, on_delete=models.CASCADE)
    content = models.CharField(max_length=200)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

TIP

단수형으로 이름을 권장하는 이유

- 단일 객체를 참조하므로, 속성명을 단수형으로 작성하면 의미가 명확해집니다.
- 코드 작성시 문맥이 자연스럽게습니다.

Migration 이후 댓글 테이블 확인

- 만들어 지는 필드 이름 규칙
 - '작성한 외래 키 필드명' + '_' + 'id'
 - 댓글 테이블의 article_id 외래 키 필드 확인
 - bigint 자료형 확인
- 참조하는 클래스 이름의 소문자(단수형)로 작성하는 것이 권장 되었던 이유



〈그림2_DB_Many To One Relationships 01_생성된 외래키 확인〉



bigint 자료형

64비트 정수형으로 약 9경까지 저장할 수 있으며, 숫자의 고갈 걱정이 없어 Primary Key(PK)를 선언할 때 해당 자료형을 사용함
외래 키는 이러한 PK 정보를 저장하기 때문에 동일하게 해당 자료형을 사용

〈개념2_DB_Many To One Relationships 01_bigint 자료형〉

※ 외래 키는 models.py의 클래스 내에 ForeignKey 필드를 작성하는 순서와 관계없이 테이블의 마지막 필드로 생성됨

댓글 생성 연습

댓글 생성 연습 준비하기

- 연습을 위한 shell 환경에서 진행
- ipython 설치 필요 (\$ pip install ipython)
- django shell 시작 후 댓글 작성을 위한 게시글 하나 작성

```
# django shell 실행 (-v 2 옵션은 선택)
$ python manage.py shell
8 objects imported automatically (use -v 2 for details).

# 게시글 생성
Article.objects.create(title='title', content='content')
```


댓글 생성 연습 (1/5)

- 댓글 생성 후 바로 저장 확인

```
# Comment 클래스의 인스턴스 comment 생성  
comment = Comment()
```

```
# 인스턴스 변수 저장  
comment.content = 'first comment'
```

```
# DB에 댓글 저장  
comment.save()
```

```
# 에러 발생
```

```
IntegrityError: NOT NULL constraint failed: articles_comment.article_id  
=> articles_comment 테이블의 ForeignKeyField, article_id 값이 저장 시 누락되었기 때문
```

※ 현재 어떤 게시글에 작성되는 댓글인지 저장될 댓글 데이터에 게시글 정보가 없는 상태

댓글 생성 연습 (2/5)

- 게시글 정보를 가져와 댓글 저장

```
# 게시글 정보 조회하여 가져오기
article = Article.objects.get(pk=1)
# 외래 키 데이터 입력
comment.article = article
# 직접적으로 테이블의 필드에 pk를 넣어도 저장되지만 권장하지 않음
# comment.article_id = article.pk

# 댓글 저장 및 확인
comment.save()
```

TIP

article_id에 직접 pk를 넣는 것을 권장하지 않는 이유

- 직접 객체를 할당하는 경우는 저장될 객체 타입을 에러 발생 유무를 통해 명확히 검증할 수 있습니다.
- 직접 pk를 입력하는 경우에는 숫자를 저장하기 때문에 잘못된 객체의 PK 값이 저장될 수 있습니다.
- 예) `comment.article_id = book.pk` # 잘못된 정보지만 에러 발생 안함

댓글 생성 연습 (3/5)

- comment 인스턴스를 통한 article 값 참조하기

```
comment.pk
# 출력: 1

comment.content
# 출력: 'first comment'

# 클래스 변수명인 article로 조회 시 해당 참조하는 게시물 객체를 조회할 수 있음
comment.article
# 출력: <Article: Article object (1)>

# 테이블 상의 article_id 값은 해당 게시글의 Primary Key(PK)값만 확인가능
comment.article_id
# 출력: 1
```

※ article_pk는 존재하지 않는 필드이기 때문에 사용 불가

댓글 생성 연습 (4/5)

- comment 를 통한 article 객체의 데이터 참조하기

```
# 1번 댓글이 작성된 게시글의 pk 조회  
comment.article.pk  
# 출력: 1
```

```
# 1번 댓글이 작성된 게시글의 content 조회  
comment.article.content  
# 출력: 'content'
```


댓글 생성 연습 (5/5)

- 두 번째 댓글 생성 및 데이터 확인

```
commentary = Comment(content='second comment', article=article)
commentary.save()
```

```
commentary.pk
```

```
# 출력: 2
```

```
commentary
```

```
# 출력: <Comment: Comment object (2)>
```

```
commentary.article.pk
```

```
# 출력: 1
```

id INTEGER	content varchar(200)	created_at datetime	updated_at datetime	article_id bigint
1	first comment			1
2	second comment			1

<그림3_DB_Many To One Relationships 01_생성된 데이터 확인>

관계 모델 참조

참조의 정의

참조

직접 대상의 정보를 저장하고 필요할 때 활용하는 것

참조는 마치 “내가 친구의 전화 번호를 저장” 하는 것과 같습니다.

내가 전화 번호를 가지고 있어 언제든지 친구에게 전화나 문자를 할 수 있습니다.

직전 실습에서 **댓글(Comment)**에는 **게시글(Article)** 정보를 저장하는 **ForeignKey** 필드인 **article**이 존재하며, 해당 필드를 통해 게시글 정보에 쉽게 접근할 수 있습니다. 그래서 **댓글(Comment)**은 **게시글(Article)**을 **참조**한다고 말합니다.

예시) 댓글의 게시글 내용에 접근: `comment.article.content`

특정 게시물(Article)의 댓글(Comment) 정보 조회하기

- QuerySet API의 .all() 사용하기 ❌

- 특정 게시물(Article)의 댓글(Comment)들이 아닌 모든 댓글 정보를 가져오게 됨

```
# 특정 게시물의 댓글이 아닌 모든 댓글을 조회함  
comments = Comment.objects.all()
```

- QuerySet API의 .filter() 사용하기

- 특정 게시물(Article) 정보를 조회 후 댓글(Comment)에서 filter를 활용해 댓글 조회 가능

```
# 특정 게시물 정보를 가져온 후 filter를 이용할 수 있음  
article = Article.objects.get(pk=1)  
comments = Comment.objects.filter(article=article)
```

※ Django는 더 직관적인 방법이 존재하며 다음 장에서 확인 가능

역참조

역참조의 정의

역참조

누가 나를 참조하는지 거꾸로 조회하는 것

역참조는 마치 “내 번호를 저장한 사람들이 누구인지 찾는 것”과 비슷합니다.

직접적으로 정보를 가지고 있지 않고 반대로 확인해야 하기 때문에 **참조하는 것을 거꾸로 찾아야 하며 이를 역참조**라고 부릅니다.

게시글(Article) 입장에서는 연결된 댓글(Comment) 정보를 담는 필드가 존재하지 않습니다.

그래서 해당 글을 참조하고 있는 댓글(Comment) 들을 역으로 찾아야 하며 Django에서는 이런 관계를 자동으로 추적해서 해당 게시글(Article)에 달린 댓글(Comment)를 쉽게 찾아올 수 있도록 **역참조 기능을 제공**합니다.

역참조 기본 구조 (1/3)

```
article.comment_set.all()
```

모델 인스턴스

related manager
(역참조 이름)

QuerySet API

1. 모델 인스턴스

- models.py 에 정의된 모델 클래스로 생성된 실제 데이터를 의미
 - 'article.title' 과 같이 속성에 접근 가능하며 속성을 수정할 수 있음
- 역참조에서는 참조 가능한 필드가 없는 모델 클래스의 인스턴스를 사용하면 됨
 - Article(1) : Comment(N) => Article 에 참조 필드가 없어서 Article의 인스턴스를 사용
 - 특정 게시글에 작성된 댓글 전체를 조회하는 요청

역참조 기본 구조 (2/3)

```
article.comment_set.all()
```

모델 인스턴스

related manager
(역참조 이름)

QuerySet API

2. related manager (역참조 이름)

- related manager 라고 부르며 N:1 혹은 N:M 관계에서 역참조 시에 사용하는 매니저를 의미
- 'objects' 매니저를 통해 QuerySet API를 사용했던 것 처럼 related manager를 통해 QuerySet API를 사용할 수 있게 됨

➤ 특정 게시글에 작성된 댓글 전체를 조회하는 요청

역참조 기본 구조 (3/3)

```
article.comment_set.all()
```

모델 인스턴스

related manager
(역참조 이름)

QuerySet API

3. QuerySet API

- 데이터를 가져오기 위한 쿼리 결과 집합을 만드는 인터페이스
- SQL 쿼리를 직접 쓰지 않고도 DB를 사용할 수 있음

➤ 특정 게시글에 작성된 댓글 전체를 조회하는 요청

related manager (역참조 이름) 이름 규칙

- 모델 클래스명 + `_set` 이 기본 값이며 Django에서 자동으로 생성해 줌
- 관계를 직접 정의하지 않은 모델에서 연결된 객체들을 조회할 수 있게 함
- 특정 댓글의 게시글 참조 (Comment → Article)
 - `comment.article`
- 특정 게시글의 댓글 목록 역참조 (Article → Comment)
 - `article.comment_set.all()`

related manager 연습 (1/2)

- django shell 실행

```
$ python manage.py shell
```

- 1번 게시글 조회

```
article = Article.objects.get(pk=1)
```

- 1번 게시글에 작성된 모든 댓글 조회하기 (역참조)

```
article.comment_set.all()
```

```
<QuerySet [<Comment: Comment object (1)>, <Comment: Comment object (2)>]>
```

related manager 연습 (2/2)

- 1번 게시글에 작성된 모든 댓글 내용 출력하기

```
# 1번 게시글의 모든 댓글 정보들을 저장
comments = article.comment_set.all()

# 1번 게시글 댓글 정보들을 반복하여 개별 댓글 내용 출력
for comment in comments:
    print(comment.content)

'''출력
first comment
second comment
'''
```

댓글 구현

댓글 CREATE

댓글 CREATE 구현 (1/12)

- 사용자로부터 댓글 데이터를 입력 받기 위한 CommentForm 정의

```
# articles/forms.py
from .models import Article, Comment

class CommentForm(forms.ModelForm):
    class Meta:
        model = Comment
        fields = '__all__'
```

댓글 CREATE 구현 (2/12)

- 댓글이 작성되는 곳은 게시글 상세(detail) 페이지 하단
- detail view 함수에서 CommentForm을 detail 페이지에서 사용할 수 있게 context로 전달

```
def detail(request, pk):  
    article = Article.objects.get(pk=pk)  
    comment_form = CommentForm()  
    context = {  
        'article': article,  
        'comment_form': comment_form,  
    }  
    return render(request, 'articles/detail.html', context)
```

댓글 CREATE 구현 (3/12)

- 댓글이 작성되는 곳은 게시글 상세(detail) 페이지 하단
- detail view 함수에서 넘어오는 CommentForm을 사용하여 detail 페이지에 렌더링

```
<!-- articles/detail.html -->  
<!-- 게시글 정보 출력 코드 -->  
<hr>
```

```
<form action="#" method="POST">  
  {% csrf_token %}  
  {{ comment_form }}  
  <input type="submit">  
</form>
```


댓글 CREATE 구현 (4/12)

- Comment 클래스의 외래 키 필드 article 또한 데이터 입력이 필요한 필드이기 때문에 출력 되는 것
- 하지만, 외래 키 필드 데이터는 **사용자로 부터 입력 받는 값이 아닌 view 함수 내에서 다른 방법으로 전달 받아 저장되어야 함**



<그림4_DB_Many To One Relationships 01_삭제가 필요한 사용자 입력>

TIP

게시글을 유저가 직접 선택 한다면 발생하는 일

- 댓글을 작성할 수 없도록 설정된 게시글을 선택해서 댓글을 추가할 수 있게 됩니다.
- 댓글을 작성하는 게시글을 자세히 확인할 수 없어 엉뚱한 게시글에 댓글을 달 수 있습니다.

댓글 CREATE 구현 (5/12)

- CommentForm의 출력 필드 조정하여 외래 키(Foreign Key) 필드가 출력되지 않도록 함

```
# articles/forms.py
from .models import Article, Comment

class CommentForm(forms.ModelForm):
    class Meta:
        model = Comment
        fields = ('content', )
```

※ fields 설정을 tuple 로 진행시 요소가 하나 일 때 마지막 콤마(,) 를 반드시 작성해야 합니다.

댓글 CREATE 구현 (6/12)

- 출력에서 제외된 외래 키 데이터는 어디서 받아와야 할까?
- detail 페이지의 URL에 게시글 정보가 존재
 - `path('<int:pk>/', views.detail, name='detail')`
 - 해당 게시글의 pk 값이 variable routing으로 전달되고 있음
- 댓글의 외래 키 데이터에 필요한 정보가 바로 게시글의 pk 값
 - 댓글 작성 시 해당 pk 값을 이용하여 게시글 데이터를 가져와 사용



variable routing

웹 주소 중 일부를 변수처럼
설정해 사용자의 입력 값에 따라
맞춤형 페이지를 보여주는 기능

<개념3_DB_Many To One Relationships 01_variable routing>

댓글 CREATE 구현 (7/12)

- 댓글 저장 로직은 detail 함수가 아닌 개별 함수로 작성
 - 댓글 저장은 게시물 상세 보기와 전혀 다른 기능이기 때문 (단일 책임 원칙)
 - 댓글 저장 시 게시물 pk 정보를 URL 로 전달하여 사용



단일 책임 원칙

함수는 하나의 기능만 수행하고,
그 기능이 변경될 이유도
하나여야 함

<개념4_DB_Many To One Relationships 01_단일 책임 원칙>

```
<!-- articles/detail.html -->
<form action="{% url 'articles:comments_create' article.pk %}" method="POST">
    {% csrf_token %}
    {{ comment_form }}
    <input type="submit">
</form>
```

```
# articles/urls.py
app_name = 'article'
urlpatterns = [
    ...
    path('<int:pk>/comments/', views.comments_create, name='comments_create'),
]
```


댓글 CREATE 구현 (8/12)

- comments_create 함수는 POST 작성만 진행
 - 댓글 작성 폼을 출력하는 GET 동작은 detail 함수에 이미 작성

```
def comments_create(request, pk):  
    article = Article.objects.get(pk=pk) # 게시글 데이터 조회  
    comment_form = CommentForm(request.POST)  
    if comment_form.is_valid():  
        comment_form.save()  
        return redirect('articles:detail', article.pk)  
    # 유효성 검사에 실패 했을 때 에러 정보 및 필요 데이터 전달  
    context = {  
        'article': article,  
        'comment_form': comment_form,  
    }  
    return render(request, 'articles/detail.html', context)
```

댓글 CREATE 구현 (9/12)

- URL 로 전달받은 게시글의 pk를 이용해서 게시글 정보를 가져옴
- 댓글 생성시 가져온 article 객체를 추가해야 함
- 가져온 article 객체는 댓글 생성시 어떻게 저장하면 될까?

댓글 CREATE 구현 (10/12)

- save 메서드의 commit 인자

save(commit=False)

- 기본적으로 commit 속성은 True가 기본 값
 - 설정 값이 True인 경우 인스턴스를 생성하고 반환한 다음 DB에도 저장 요청을 보냄
- commit이 False 인 경우 DB에 저장 요청을 보내지 않고 인스턴스만 반환
 - Create, but don't save the new instance.
- 댓글을 저장할 때 바로 DB에 저장 요청을 보내는 것이 아닌 게시글 정보를 추가한 후 저장 요청을 보내도록 로직을 구성하면 게시글 정보와 함께 댓글을 저장 할 수 있게 됨

댓글 CREATE 구현 (11/12)

```
# articles/views.py
def comments_create(request, pk):
    article = Article.objects.get(pk=pk)
    comment_form = CommentForm(request.POST)
    if comment_form.is_valid():
        comment = comment_form.save(commit=False)
        comment.article = article
        comment.save()
        return redirect('articles:detail', article.pk)
    # 유효성 검사에 실패 했을 때 에러 정보 및 필요 데이터 전달
    context = {
        'article': article,
        'comment_form': comment_form,
    }
    return render(request, 'articles/detail.html', context)
```

댓글을 저장하지 않고 인스턴스만 생성
댓글에 게시글 정보 추가
댓글 정보 DB에 저장 요청

댓글 CREATE 구현 (12/12)

- 댓글 작성 후 DB에서 생성 확인
 - article_id 필드에 게시물 정보가 저장되어 있는 것을 확인

id INTEGER	* content varchar(200)	* created_at datetime	* updated_at datetime	* article_id bigint
1	first comment			1
2	second comment			1
3	1번 게시물 댓글 작성!			1

<그림5_DB_Many To One Relationships 01_저장된 댓글 데이터 확인>

댓글 READ

댓글 READ 구현 (1/2)

- 댓글이 보이는 위치는 게시물 상세 페이지 하단 위치
 - detail view 함수에서 전체 댓글 데이터를 조회해서 detail.html 로 전달하여 댓글 데이터 조회

```
# articles/views.py
def detail(request, pk):
    article = Article.objects.get(pk=pk)
    comment_form = CommentForm()
    # 특정 게시글의 댓글이 아닌 모든 댓글을 조회하기에 사용 하지 않음
    # comments = Comment.objects.all()           # 잘못된 댓글 정보 조회
    comments = article.comment_set.all()        # 특정 게시글의 댓글을 역참조로 조회
    context = {
        'article': article,
        'comment_form': comment_form,
        'comments': comments,
    }
    return render(request, 'articles/detail.html', context)
```

댓글 READ 구현 (2/2)

- 댓글 출력 확인

```
<a href="#">UPDATE</a>

<h4>댓글 목록</h4>
<ul>
  {% for comment in comments %}
    <li>{{ comment.content }}</li>
  {% endfor %}
</ul>
```

1 번째 글

제목: title

내용: content

작성 시각: 2022. 11. 12. 12:00

수정 시각: 2022. 11. 12. 12:00

[UPDATE](#)

[DELETE](#)

댓글 목록

- first comment
- second comment
- 1번 게시글 댓글 작성!

Content:

[제출](#)

[\[back\]](#)

댓글 DELETE

댓글 DELETE 구현 (1/4)

- 개별 댓글마다 삭제할 수 있도록 기능을 추가
- 어떤 댓글을 삭제해야 하는지 삭제 대상 정보를 전달하기 위해 variable routing 이용

```
# articles/urls.py
app_name = 'articles'
urlpatterns = [
    ...,
    path(
        '<int:article_pk>/comments/<int:comment_pk>/delete/',
        views.comments_delete,
        name='comments_delete'
    ),
]
```

- ※ article_pk 정보는 URL 패턴을 일관되게 작성하기 위함 외에도 삭제 후 detail 페이지로 돌아갈 때 사용할 수 있는 정보이기 때문에 작성합니다.

댓글 DELETE 구현 (2/4)

- 댓글 삭제 view 함수 정의

```
# articles/views.py

def comments_delete(request, article_pk, comment_pk):
    comment = Comment.objects.get(pk=comment_pk)
    comment.delete()
    # 댓글 삭제 후 article 게시물 상세 페이지로 이동
    return redirect('articles:detail', article_pk)
```

댓글 DELETE 구현 (3/4)

- 댓글 삭제 버튼 추가
 - variable routing으로 전달되는 게시글 pk, 삭제될 댓글 pk 전달

```
<!-- articles/detail.html -->
<h4>댓글 목록</h4>
<ul>
  {% for comment in comments %}
    <li>
      {{ comment.content }}
      <form action="{% url 'articles:comments_delete' article.pk comment.pk %}" method="POST">
        {% csrf_token %}
        <input type="submit" value="DELETE">
      </form>
    </li>
  {% endfor %}
</ul>
```


댓글 DELETE 구현 (4/4)

- 댓글 삭제 버튼 출력 확인 및 삭제 테스트

댓글 목록

- first comment
DELETE
- second comment
DELETE
- 1번 게시글 댓글 작성!
DELETE

댓글 목록

- first comment
DELETE
- 1번 게시글 댓글 작성!
DELETE

<그림7_DB_Many To One Relationships 01_댓글 삭제 가능>

참고

데이터 무결성

데이터 무결성이란

- 데이터베이스에 저장된 데이터의 정확성, 일관성, 유효성을 유지하는 것
- 데이터베이스에 저장된 데이터 값의 정확성을 보장하는 것
- 중요성
 - 데이터의 신뢰성 확보
 - 시스템 안정성
 - 보안 강화

admin site 댓글 등록

admin site에 댓글 등록하기

- 작성된 Comment 모델에 대해 Admin site 에서 관리할 수 있도록 아래와 같이 등록

```
# articles/admin.py
from .models import Article, Comment

admin.site.register(Article)
admin.site.register(Comment)
```

Site administration

ARTICLES

Articles

+ Add ✎ Change

Comments

+ Add ✎ Change

<그림8_DB_Many To One Relationships 01_admin 페이지에 등록된 댓글 모델>

댓글 추가 구현

1. 댓글이 없는 경우 대체 콘텐츠 출력

- DTL 의 'for empty' 태그 활용
 - for 문에 반복할 요소가 없는 경우 empty 태그가 실행됨

```
<!-- articles/detail.html -->
{% for comment in comments %}
  <li>
    {{ comment.content }}
    <form action="{% url 'articles:comments_delete' article.pk comment.pk %}" method="POST">
      {% csrf_token %}
      <input type="submit" value="DELETE">
    </form>
  </li>
{% empty %}
  <p>댓글이 없어요..</p>
{% endfor %}
```


2. 댓글 개수 출력하기

- DTL filter 의 'length' 활용

```
<!-- views.py에서 전달받은 댓글로 길이 확인-->
{{ comments|length }}
```

```
<!-- 역참조를 이용하여 댓글 길이 확인 -->
{{ article.comment_set.all|length }}
```

- QuerySet API 의 count() 메서드 활용

```
<!-- 역참조로 QuerySet API 이용 댓글 길이 확인 -->
{{ article.comment_set.count }}
```

Many to one relationships

- 1901. 도서 관리 서비스 만들기 - 1:N 관계 정의
- 1896. 레스토랑 관리 서비스 - 모델 정의
- 3055. 댓글 생성 기능 구현 준비

관계 모델 참조

- 1902. 도서 관리 서비스 - 1:N 관계 데이터 생성
- 1898. 레스토랑 관리 서비스 - 메뉴

댓글 구현

- 3056. 댓글 생성 기능 구현
- 3057. 댓글 조회 및 삭제 기능 구현
- 1899. 레스토랑 관리 서비스 - 메뉴 상세정보와 수정
- 1900. 레스토랑 관리 서비스 - 이미지 추가와 코드 개선

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

- 1 related_name이 없을 때, 올바른 역참조 코드는 무엇인가요?

```
# models.py
class Article(models.Model):
    pass
class Comment(models.Model):
    article = models.ForeignKey(Article, ...)

article = Article.objects.get(id=1)
```

- a) article.comments.all()
- b) article.comment_set.all()
- c) Comment.objects.get(article)
- d) article.Comment.all()

- 2 id가 1인 게시물에 댓글을 생성하는 올바른 방법은?

```
article = Article.objects.get(id=1)
content = '새 댓글'
```

- a) Comment.objects.create(...)
- b) Comment.objects.new(...)
- c) Comment.create(...)
- d) article.comments.add(...)

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

3 다음 문장이 올바른지 O, X로 답하십시오.

'N'쪽 객체에서 '1'쪽 객체 필드 접근은 '역참조'이다.

5 admin 사이트에 모델을 등록하는 함수는 무엇인가요?

```
# admin.py
from django.contrib import admin
from .models import Comment

# ???(Comment)
```

4 Django 템플릿에서 댓글이 없는 경우를 처리하기 위해 사용하는 문법은 무엇인가요?

- a) {% if not comments %}
- b) {% empty %}
- c) {% else %}
- d) {% forempty %}

6 다음 중 N:1 관계를 설명한 것은?

- a) 하나의 댓글은 여러 게시글에 달린다.
- b) 하나의 게시글에 여러 댓글이 달린다.
- c) 댓글과 게시글은 1:1 관계이다.
- d) 댓글은 게시글 없이도 존재한다.

Q 각 문제에 올바른 답안을 생각해봅시다

7 댓글 모델에서 게시글을 참조할 때 사용하는 필드는?

- a) OneToOneField
- b) ManyToManyField
- c) ForeignKey
- d) CharField

9 다음 문장이 올바른지 O, X로 답하시오.

ForeignKey로 연결된 모델은 역참조 시 자동으로 `related_name` 속성이 생성된다.

8 다음 문장이 올바른지 O, X로 답하시오.

Django shell에서는
`Comment.objects.create(post=post, content="내용")`
으로 댓글을 생성할 수 있다.

10 Django 댓글 생성 시 `commit=False` 사용 이유는?

- a) DB 저장 없이 객체 삭제
- b) 생성 후 필드 추가 설정
- c) 즉시 저장해 성능 향상
- d) 조건에 맞는 객체 필터링

Q 각 문제에 올바른 답안을 생각해봅시다

11 다음 중 N:1 관계에 해당하지 않는 예시는 무엇인가요?

- a) 여러 명의 학생이 한 명의 강사에게 배운다.
- b) 여러 개의 댓글이 하나의 게시글에 달린다.
- c) 여러 개의 주문이 한 명의 고객에게 속한다.
- d) 여러 명의 배우가 여러 영화에 출연한다.

13 다음 문장이 올바른지 O, X로 답하시오.

댓글(Comment) 모델에서 게시글(Article)을 참조하도록 ForeignKey를 설정하면, 게시글에서 직접 참조 필드를 통해 댓글 목록을 조회할 수 있다.

12 ForeignKey 필드 필수 속성 2가지는? (주관식)

14 다음 중 ForeignKey 필드명을 작성할 때 권장되는 방식은 무엇인가요?

- a) 복수형으로 작성한다. (예: articles)
- b) 참조하는 모델의 클래스명을 그대로 쓴다. (예: Article)
- c) 참조하는 모델의 클래스명을 소문자 단수형으로 쓴다.
- d) 필드명은 아무렇게나 작성해도 된다.

확인 문제



정답 및 해설

1. b) `article.comment_set.all()` / 2. a) `Comment.objects.create(...)` / 3. X / 4. b) `{% empty %}` / 5. `admin.site.register`
6. b) 하나의 게시글에 여러 댓글이 달린다. / 7. c) `ForeignKey` / 8. O / 9. X

1. `ForeignKey`에서 `related_name`을 별도로 설정하지 않으면, Django는 자동으로 '참조하는모델명_set' (소문자) 형태의 역참조 매니저를 생성합니다.
2. `create()` 메서드를 사용하며, `ForeignKey` 필드는 객체 또는 '_id' 접미사를 붙인 pk 값으로 지정할 수 있습니다.
3. 'N'쪽에서 '1'쪽으로 접근하는 것은 '참조'이며, '1'쪽에서 자신을 참조하는 'N'쪽 객체 목록에 접근하는 것이 '역참조'입니다.
4. `{% for comment in comments %} ... {% empty %} ... {% endfor %}` 형태로 사용하면, 댓글이 없을 때 대체 콘텐츠를 자연스럽게 출력할 수 있습니다. Django의 `for-empty` 문법은 사용자 경험을 고려한 표현입니다.

5. `admin.py`에서 `admin.site.register()` 함수를 사용하여 모델을 관리자 페이지에 등록할 수 있습니다.
6. N:1 관계는 여러 댓글이 하나의 게시글을 참조하는 구조입니다.
7. `ForeignKey`는 N:1 관계를 정의할 때 사용하는 필드입니다.
8. Django shell에서는 ORM을 통해 객체를 직접 생성할 수 있으며, `ForeignKey` 필드에는 참조할 객체(post)를 전달해야 합니다.
9. `related_name`은 명시적으로 설정하지 않으면 자동으로 생성되지 않습니다. 대신 Django는 기본적으로 모델명_set 형식의 역참조 매니저를 제공합니다.

확인 문제



정답 및 해설

10. b) 생성 후 필드 추가 설정 / 11. d) 여러 명의 배우가 여러 영화에 출연한다. / 12. 'model class, on_delete' / 13. X
14. c) 참조하는 모델의 클래스명을 소문자 단수형으로 쓴다.

10. `form.save(commit=False)`는 폼 데이터를 기반으로 객체를 생성하되, 데이터베이스에는 저장하지 않습니다. 이 시점에서 객체를 수정하거나 관계 필드(post, user 등)를 설정한 후, `.save()`를 호출하여 저장할 수 있습니다. N:1 관계에서 외래 키를 설정할 때 자주 사용됩니다.
11. d의 경우는 N:M 관계입니다. 배우는 여러 영화에 출연할 수 있고, 영화도 여러 배우를 가질 수 있기 때문입니다. N:1 관계는 여러 개의 레코드가 하나의 레코드와 연결되는 구조입니다.
12. ForeignKey 필드를 정의할 때 참조할 모델 클래스(to)와 참조 대상이 삭제될 때 처리 방식을 지정하는 `on_delete`는 필수로 작성해야 합니다.

13. 게시글(Article) 모델에는 댓글(Comment)을 직접 참조하는 필드가 없기 때문에, 직접 접근할 수 없습니다. 대신 Django는 역참조 기능을 통해 ``article.comment_set.all()``과 같은 방식으로 참조하는 데이터를 거꾸로 조회할 수 있도록 지원합니다.
14. ForeignKey 필드명은 참조하는 모델을 단일 객체로 가리키므로, 의미가 명확하도록 소문자 단수형으로 작성하는 것이 좋습니다. 예: ``article = models.ForeignKey(...)``

활동 정리

핵심 키워드

개념	설명	예시
ForeignKey	다른 모델과 N:1 관계를 정의할 때 사용하는 필드	<code>comment = models.ForeignKey(Article, on_delete=...)</code>
Many to One	여러 개의 하위 객체가 하나의 상위 객체를 참조하는 모델 관계	여러 댓글 → 하나의 게시글
ORM	Django에서 데이터베이스를 객체처럼 다루는 방식	<code>Comment.objects.create(...)</code>
역참조	상위 모델에서 연결된 하위 모델을 조회하는 방식	<code>article.comment_set.all()</code>
admin 등록	admin 사이트에서 모델을 시각적으로 관리하도록 설정	<code>admin.site.register(Comment)</code>
for-empty 문법	반복문에서 데이터가 없을 경우를 처리하는 Django 템플릿 문법	<code>{% for comment in comments %}...{% empty %}...{% endfor %}</code>
데이터 무결성	모델 관계 설정 시 데이터 일관성과 안정성을 유지하는 설계 방법	<code>on_delete=models.CASCADE</code>

<표3_DB_Many To One Relationships 01_핵심 키워드>

활동 정리

오늘 공부한 내용 요약 및 정리

- N:1 관계는 여러 개의 댓글이 하나의 게시글에 연결되는 구조이며, Django에서는 ForeignKey를 사용해 구현합니다.
- Django shell을 활용한 ORM 실습을 통해 관계형 데이터 생성과 조회를 연습했습니다.
- 댓글 기능 구현(CREATE, READ, DELETE)을 통해 실제 웹 애플리케이션의 동작 원리를 파악했습니다.
- 역참조 기능을 통해 게시글에서 관련 댓글을 불러올 수 있으며, admin 페이지를 활용한 댓글 관리도 실습했습니다.
- for-empty 문법을 적용하여 댓글이 없을 경우도 고려한 UX 개선을 수행했습니다.

활동 정리

사내 게시판에 작성된 게시글에 댓글을 작성할 수 있도록 만들어 봅시다.

- 사용자가 게시글에 댓글을 달 수 있는 기능을 Django로 쉽게 구현할 수 있습니다.
- 게시글과 댓글을 관계를 설정하여 데이터베이스에 저장할 수 있습니다.

1. 게시글과 댓글은 **N:1** 관계이므로, 모델 간 관계 설정이 필요해 **ForeignKey**를 사용해 댓글 모델을 정의해야 합니다.
2. **역참조** 개념을 활용하면 게시글에서 관련된 댓글을 쉽게 조회할 수 있습니다.
3. **for-empty** 문법을 사용하면 댓글이 없는 경우에도 자연스러운 UI를 구현할 수 있습니다.

- admin 페이지에 댓글을 등록하고 관리하여 쉽게 관련 데이터를 관리할 수 있습니다.
- N:1 관계 설정을 이용해서 누가 작성한 게시글 혹은 댓글인지 만들어 볼 수 있습니다.

감사합니다

